

PROJECT OS — V2 (handoff ChatGPT web)

Objectif de ce document : fournir à ChatGPT l'état RÉEL et complet de Project OS afin qu'il génère le **prompt de continuation** de la roadmap en 30 phases de développement. Le goal final (`/goal`) = **PROJECT_OS_PRODUCTION_READY** = **PASS** après les 30 phases. **Le code, les tests et le runtime font autorité.** Aucune fonctionnalité n'est inventée.

Project OS — Référence V2 (v0.1.0)

TypeScript/Node VS Code extension C++17 CLI LocalAI (loopback)

1. Contexte & chemins (réels)

Élément	Valeur réelle
Racine (repo)	<code>C:\Users\eiden\Desktop\dev\projet-os</code>
Projets gérés (hub-spoke)	<code>C:\Users\eiden\Desktop\dev\projects\<slug></code>
Endpoint LocalAI canonique	<code>http://127.0.0.1:8080/v1</code>
CLI C++	<code>C:\Users\eiden\Desktop\dev\projet-os\cli-cpp\cmake-build\project-os-cli.exe</code>
Passerelle Node	<code>bin\project-os-bridge.mjs</code> (protocole v2)
Toolchain C++	<code>C:\msys64\mingw64\bin</code> (g++ 16.1, cmake)
Node / npm	v24.16.0 / 11.13.0
Typescript	^5.5.4 · module Node16 · target ES2022

2. Architecture (hub-spoke, à préserver)

```
UI / extension (VS Code)
  → application / domain (src/*, pur, sans vscode)
    → services
      → providers / adapters
        → LocalAI (loopback) / filesystem / Node bridge / GPU
```

Couche	Rôle
Control plane	Extension VS Code (commandes, arbres, statut, config)
Domain pur	Logique testable, aucun <code>vscode</code> import
Moteur IA	LocalAI openai-compatible en loopback (no cloud fallback)
Projets enfant	Workspaces auto-gouvernés (goal/todo/addons/autonomy/artifacts)
CLI C++	Client de premier niveau (menus) → bridge Node

3. État RÉEL (mesuré — autorité)

3.1 Baseline

Mesure	Valeur réelle	Statut
typecheck	<code>tsc -p ./ --noEmit</code>	0 erreur
tests Node (<code>npm test</code>)	293/293 (48 fichiers <code>*.test.ts</code>)	PASS
tests C++ (ctest)	pos_json_test 1/1	PASS
build C++ (cmake)	project-os-cli + pos_json_test	exit 0
source .ts	140 fichiers (dist 140)	—

3.2 LocalAI

Endpoint	<code>GET /v1/models</code> HTTP 200	Fonctionnel
Modèles listés	15 (dont granite-4.2-3b-flash, ministral-3b-flash, granite-4.0-micro-flash, qwen3-4b, qwen3-1.7b, smollm3-3b, deepseek-r1-distill-qwen-1.5b)	Inventaire réel
Preflight V2	latence + inventoryShape (openai/ollama) + health	PASS

3.3 GPU (réel, maintenant débloqué)

GPU	NVIDIA GeForce RTX 5060 Laptop, driver 616.56
VRAM total / libre	8151 / ~537-605 MiB (modèle chargé)
Modèle qualifié FLASH_READY	<code>granite-4.2-3b-flash</code> (Q4_K_M, llama-cpp)
GPU_OFFLOAD_PROOF	PASS — delta VRAM -3043 MiB (3648→605), smoke <code>content="4"</code> HTTP 200
Benchmark réel	TTFT 379.3 ms · 63.3 tok/s · VRAM peak 7546 MiB · correctness 1 · stability 0.98 · passes true
Endurance GATE_1 5-min	PASS — 114 inf GPU, 0 bug, continuité stable (ladder <code>completed=[5]</code>)

Le modèle est en mode **reasoning** : le `content` n'apparaît qu'avec assez de `max_tokens` (le raisonnement remplit d'abord le budget). De plus `/backend/monitor` renvoie 500 (différent de signature) — la preuve de résidence GPU repose sur le delta VRAM + smoke + benchmark.

4. Commandes & parité

Capacité	Slash /	VS Code	CLI C++	Test
create project	<code>/create</code>	<code>clineProjectOS.create</code>	menu	<code>projectFactory.test</code>
set goal	<code>/goal</code>	<code>clineProjectOS.goal</code>	menu	<code>slashCommand.test</code>
todo	<code>/todo</code>	<code>via /slash</code>	menu	<code>todoCommand.test</code>
addon	<code>/addon</code>	<code>clineProjectOS.addon</code>	menu	<code>addonManager.test</code>
autonomy	<code>/autonomy</code>	<code>clineProjectOS.autonomy</code>	menu	<code>autonomy.test</code>
docs online	<code>/docs</code>	<code>via /slash</code>	menu	<code>docsNavigator.test</code>
project status	<code>/project</code>	<code>clineProjectOS.project.status</code>	<code>slash raw</code>	<code>productCommands.test</code>

tokens+cost report	/report (export)	clineProjectOS.tokens.*	—	usageReport.test
--------------------	------------------	-------------------------	---	------------------

Autres commandes VS Code : controlcenter.open, artifact.create/open/search, agent.start/abort, provider.preflight, project.scan, workspace.report, mode.route, budget.status, antigravity.verifyWorkspace, projects.refresh. → **22 commandes** `clineProjectOS.*` .

5. Invariants NON-NÉGOCIABLES (à préserver)

- **GPU** : `allowCpuFallback=false` , `requireGpu=true` , `requireFlashReady=true` pour les gates concernés ; GPU_OFFLOAD_PROOF doit être une preuve réelle (delta VRAM + backend + smoke) ; jamais de qualification/PASS/benchmark simulé.
- **Process** : jamais de kill auto de process utilisateur, jamais de reset Docker, jamais de restart Windows ; seule la protection IDE (Antigravity) avec conditions validées.
- **LocalAI** : loopback canonique `127.0.0.1:8080/v1` ; pas de fallback cloud caché.
- **Security** : `trust_remote_code=false` ; SHA/empreintes vérifiées ; pas de secret dans les logs ; protéger .env/clés/.git/config ; pas de path traversal ; pas d'écriture hors workspace ; delete/rename interdits par défaut ; pas d'élévation destructive.
- **Honesty** : états autorisés PASS / PASS_WITH_WARNINGS / WARN / BLOCKED / FAIL / NOT_TESTED / NOT_APPLICABLE ; interdits : « probablement PASS », « devrait fonctionner », « semble validé ».

6. Modules & fonctionnalités (implémentés & testés)

Domaine	Modules	Rôle
Projets	ProjectFactory (transactionnel + rollback), ManagedProjectRegistry, GoalProofEngine, TodoTracker, AddonManager(verifyLock)	Création/registre/goal par preuve/todo/addons
Autonomie	AutonomyService/Runner, AutonomyWriteScope (guardWritePath), AutonomyWriteLane, AutonomyActivityLedger, AutonomyModelSelector	Plan/run gated + write-lane borné
Routing	IntelligentModelRouter, TaskClassifier (8 classes), ModelCandidate, ModelScoreboard, ModelCatalogService, DeterministicRouter, EnduranceRouter	8 politiques, budget-gated, déterministe
Budget/Coût	ProjectBudgetGovernor (réservations atomiques), CostModel/Ledger, BudgetStatus, BudgetSimulator	Plafond PAOG, LocalAI EXACT_ZERO
Tokens	TokenLedger, UsageReport, UsageObservation, WorkspaceRegistry, HistoricalUsagelImporter, tokenEstimate	Télémétrie exacte + coût + perf
Artifacts	ArtifactRegistry, ArtifactStateMachine (11 statuts), ArtifactStore(atomic+drop), ArtifactAudit	26 types, versioning, SHA256
GPU/Endurance	GpuEvidence, GpuBenchmark, FleetQualifier, GpuQualificationQueue, EnduranceLadder (5/10/20/30/60), EnduranceLedger/Reset/Failover, EnduranceLab	Preuve offload + benchmark + ladder
Qualité	RegressionEngine, WorkspaceDrift, FailureRecovery	Anti-régression, drift, récupération
Docs/Sec	DocsNavigator, DocsCodingAdviser, DocsCheckPrewrite, StaticPostGen, WriteLaneGuard, ProviderPolicyValidator, OfficialDocsGuard	Doc officielle + guards
Observabilité	StructuredLogger, ProductionLogger(correlation), DashboardRenderer, LiveDashboard, TextOutputFormatter	Logs structurés + dashboard temps réel
UI	ControlCenterProvider, ControlCenterCockpit, ArtifactsTreeProvider, ProjectsTreeProvider	Dashboard + arbres + cockpit evidence

Workspace	WorkspaceTopology, WorkspaceTrustGuard, AntigravityIDERestartGuard, AntigravityProcessResolver	Confiance/topologie/IDE guard
Runtime	ProviderProfile, ProviderPreflight, ClineRuntimeAdapter, RuntimeEventNormalizer, StreamingBuffer, InferenceEvidence	Provider + adaptateur Cline SDK
CLI C++	pos_json.hpp, pos_model.hpp, pos_runner.hpp, project_os_cli.cpp, test_pos.cpp	Menus + parse JSON + bridge v2
Bridge	project-os-bridge.mjs (protocole v2 : protocol/requestId/timing/exitCode)	Dispatch réel vers registre slash

7. État de la roadmap 30 phases

Phase	Gate	Statut réel
01 Baseline forensique	BASELINE_LOCKED	PASS
02 Config canonique & provider	PROVIDER_POLICY_CANONICAL	PASS
03 LocalAI preflight V2	LOCALAI_PREFLIGHT_V2	PASS
04 GPU Evidence Engine	GPU_EVIDENCE_ENGINE	PASS
05 Regression Engine	REGRESSION_ENGINE	PASS
06 Project Factory V2	PROJECT_FACTORY_V2	PASS
07 Goal Engine V2	GOAL_ENGINE_V2	PASS
08 Slash Platform V2	SLASH_PLATFORM_V2	PASS
09 Command Parity	COMMAND_PARITY	PASS
10 Control Center V2	CONTROL_CENTER_V2	PASS
11 Artifact V2	ARTIFACT_V2	PASS
12 Addon Manager V2	ADDON_MANAGER_V2	PASS
13 Docs Navigator/Guard V2	DOCS_GUARD_V2	PASS (modules déjà testés)
14 Project DNA V2	PROJECT_DNA_V2	PASS
15 Workspace Drift	WORKSPACE_DRIFT	PASS
16 Autonomy Planner V2	AUTONOMY_PLAN_V2	PASS
17 Write-Lane Security V2	WRITE_LANE_SECURITY_V2	PASS
18 Autonomy Runner V2	AUTONOMY_RUNNER_V2	PASS
19 Model Catalog V2	MODEL_CATALOG_V2	PASS
20 Router V2	ROUTER_V2	PASS
21 Budget V2	BUDGET_V2	PASS
22 Token Intelligence V2	TOKEN_INTELLIGENCE_V2	PASS
23 Observability V2	OBSERVABILITY_V2	PASS
24 Endurance Ladder V2	ENDURANCE_LADDER_V2	PASS (harness) / GATE_1 5-min PASS

25 GPU Benchmark Fleet V2	GPU_FLEET_V2	PASS (qualification réelle granite-4.2)
26 C++ CLI V2	CPP_CLI_V2	PASS
27 Node Bridge V2	NODE_BRIDGE_V2	PASS
28 Security & Recovery	RECOVERY_SECURITY	PASS
29 Performance / Scale	PERFORMANCE_SCALE	PASS (mesures réelles)
30 Production Readiness	FINAL_RELEASE_GATE	PENDING — rungs 10/20/30/60 restants à valider en GPU réel

8. Protocole de MAJ : 30 étapes (à appliquer pour CHAQUE phase)

Chaque phase est une MAJ qui doit passer ces 30 étapes (aucune supprimée ; NOT_APPLICABLE avec justification si non pertinente).

```

01 Reconnaissance (lire package.json/tsconfig/CMake/src/tests/cli-cpp/bin/artifacts)
02 Git/Snapshot (branche/HEAD/status ; checkpoint logique – pas de .git dans ce snapshot)
03 Baseline tests AVANT (typecheck, unit, integration, C++, bridge)
04 Cartographie d'impact (modules/API/commands/settings/files/tests/artifacts)
05 Analyse de risque (LOW/MEDIUM/HIGH/CRITICAL ; fs, security, provider, GPU, routing, budget, state
machine, concurrency, migration, CLI, VS Code)
06 Vérification invariants (loopback LocalAI, no CPU fallback, trust_remote_code=false, write guards,
approval gates, no destructive process actions)
07 Preflight LocalAI (si inference) – GET /v1/models réel, health, streaming, erreurs ; jamais inventer
08 Preflight GPU (si GPU/modèles/endurance) – nvidia-smi, VRAM, backend, preuve d'offload ; no kill
09 Audit docs officielles (docs officielles > code source > changelogs > issues)
10 Change contract (GOAL/SCOPE/NON-GOALS/FILES/RISKS/INPUTS/OUTPUTS/ACCEPTANCE/ROLLBACK/TEST PLAN/SECURITY)
11 Plan de diff minimal (patch local, fonction pure, adapter, service isolé, migration compatible)
12 Tests de spécification (nominal + edge)
13 Modification domain (logique métier dans src/*, sans vscode)
14 Adapter/infrastructure (fs, LocalAI, ledger, registry, provider, bridge, GPU)
15 Intégration extension VS Code (si applicable)
16 Parité slash command (slash + VS Code + CLI ; matrice)
17 Parité CLI C++ (pos_json/model/runner/project_os_cli ; ne pas casser l'existant)
18 Bridge Node (durcir : schema, exit code, stdout/stderr, JSON, quoting, timeout, error typing)
19 Tests unitaires (nominal, invalid, boundary, null/empty, exceptions, état persistant, transitions)
20 Tests intégration (connexions réelles inter-couches ; pas de mock quand la preuve runtime est le but)
21 Tests négatifs/security (path traversal, secret, protected, invalid slug/transition, malformed JSON,
invalid model, unavailable provider, CPU fallback, expired approval/reservation, concurrent writes)
22 Benchmark avant/après (durée, mémoire, VRAM, TTFT, tokens/s, I/O, startup, refresh, bridge latency) –
jamais « plus rapide » sans mesure
23 Observabilité (StructuredLogger, ProductionLogger, eventNormalizer, InferenceEvidence, ledgers ; no
secrets)
24 Artefacts/preuves (implementation_plan, change_contract, code_diff, test_report, benchmark,
security/performance/qa_report, checkpoint ; via ArtifactRegistry)
25 Build complet (Node, extension, C++, bridge ; capture exit codes)
26 Full test suite (comparer pré/post ; pas de baisse silencieuse du count)
27 Regression diff (BEFORE/AFTER : tests, perf, commands, settings, artifacts, API, security invariants)
28 Failure/recovery test (un scénario d'échec réaliste ; récupération, état persistant, ledger, rollback,
erreur affichée, pas de faux PASS)
29 Documentation + version (README, docs, CHANGELOG, feature matrix)
30 Gate de sortie (PHASE, STATUS, HEAD (n/a), FILES_CHANGED, TESTS_BEFORE, TESTS_AFTER, NEW_TESTS,
REGRESSIONS, GPU_PROOF, LOCALAI_PROOF, SECURITY, PERFORMANCE, ARTIFACTS, KNOWN_LIMITATIONS, NEXT_ACTION)
– Ne pas commencer une phase suivante si FAIL/BLOCKED.

```

9. GOAL FINAL & HANDOFF

/goal atteint quand : `PROJECT_OS_PRODUCTION_READY = PASS` .

Condition : **Toutes** les gates indispensables réelles et vertes — dont la ladder d'endurance GPU 5/10/20/30/60 min complète (GATE_1 5-min déjà **PASS** ; 10/20/30/60 restants).

Ce qui reste exactement (bloqueur unique) : terminer les rungs GPU 10 → 60 min (105 min de durée cumulée) sur le modèle qualifié, puis le pack release final.

10. Format de sortie attendu par MAJ (style Cline)

```
┌── PROJECT OS – PHASE XX / 30 ──┐
STATUS / HEAD BEFORE→AFTER / FILES_CHANGED / TESTS BEFORE→AFTER
DATAS (chiffres réels)
GPU_PROOF / LOCALAI_PROOF / SECURITY / BENCHMARK / ARTIFACTS / KNOWN_LIMITATIONS
NEXT_ACTION
└──────────────────────────────────┘
[1] PREUVES [2] TESTS [3] DIFF [4] BENCHMARK [5] RISQUES [6] ARTEFACTS [7] NEXT ACTION
```

11. Priorité de développement

1. CORRECTNESS · 2. SECURITY · 3. EVIDENCE · 4. REGRESSION SAFETY · 5. RECOVERY · 6. OBSERVABILITY · 7. PERFORMANCE · 8. UX · 9. NEW FEATURES

Une feature qui augmente la surface mais réduit la fiabilité est refusée.

Limites documentées (honnêteté) : VRAM 8 Go → un backend actif à la fois ; granite-4.2 est un modèle de raisonnement (content seulement avec assez de max_tokens) ; /backend/monitor renvoie 500 (signature mismatch) ; licence DeepSeek UNKNOWN ; catalog modèle reporte UNKNOWN pour les champs non déclarés ; ladder 5/5 précédent archivé (ladder-state.backup-5of5.json), cycle relancé à GATE_1.